



✓ Predict Customer Churn with Python

Table of Contents

- 1 Introduction
- 2 Import libraries
- 3 Read dataset
- 4 Exploratory data analysis (EDA)
 - 4.1 Shape of dataset
 - 4.2 Preview dataset
 - 4.3 Summary of dataset
 - 4.4 Statistical properties of dataset
- 5 Feature selection
- 6 Convert categorical columns to numeric columns
 - 6.1 Explore Gender variable
 - 6.2 Explore Geography variable
- 7 Feature Scaling
- 8 Model Training
 - 8.1 Predict accuracy with different algorithms
 - 8.2 Plot the classifier accuracy scores
- 9 Feature Importance
 - 9.1 Feature importance with Random Forest model
 - 9.2 Drop least important feature
- 10 Confusion Matrix
- 11 Classification Metrics
 - 11.1 Classification Report
 - 11.2 Classification Accuracy
 - 11.3 Classification Error
 - 11.4 Precision
 - 11.5 Recall
 - 11.6 True Positive Rate
 - 11.7 False Positive Rate
 - 11.8 Specificity (True Negative Rate)
 - 11.9 f1 – score
 - 11.10 Support
- 12 Cross-Validation
- 13 Results and Conclusion
- 14 References

1. Introduction

- **Customer churn** (also known as customer attrition) is the loss of clients or customers. It is most commonly expressed as the percentage of service subscribers who discontinue their subscriptions within a given time period. In this project, we will build a machine learning model to predict customer churn.

✓ 2. Import libraries

- We will start off by importing the required libraries.

```

import numpy as np
import pandas as pd
import sklearn
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV , KFold , cross_val_score

# Classification algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.gaussian_process.kernels import RBF
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier, VotingClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis , QuadraticDiscriminantAnalysis
from sklearn.neural_network import MLPClassifier
import lightgbm as lgb
import xgboost as xgb

# Classification metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, precision_score, recall_score, f1_score

```

3. Read dataset

```
df=pd.read_csv('/content/Churn_Modelling.csv')
```

4. Exploratory Data Analysis

- Now, we will perform EDA to gain insights about our data.

4.1 Shape of dataset

```
df.shape
```

 (10000, 14)

- Our dataset contains 10000 instances and 14 variables.
- Now, let's take a quick look of our dataset.

4.2 Preview dataset

```
df.head()
```



	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

4.3 Summary of dataset

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   RowNumber             10000 non-null  int64
 1   CustomerId            10000 non-null  int64
 2   Surname               10000 non-null  object
 3   CreditScore           10000 non-null  int64
 4   Geography             10000 non-null  object
 5   Gender               10000 non-null  object
 6   Age                  10000 non-null  int64
 7   Tenure               10000 non-null  int64
 8   Balance              10000 non-null  float64
 9   NumOfProducts        10000 non-null  int64
10   HasCrCard            10000 non-null  int64
11   IsActiveMember       10000 non-null  int64
12   EstimatedSalary      10000 non-null  float64
13   Exited               10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

- We can see that there are 3 categorical variables and 11 numerical variables in the dataset.
- Also, there are no missing values in the dataset.

4.4 Statistical properties of dataset

```
df.describe()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	Exited
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000	10000.000000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100	0.515100
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797	0.499797
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000	0.000000
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000	0.000000
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000	1.000000
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000	1.000000
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000	1.000000

5 Feature selection

- We can see that there are 14 variables in the dataset.
- The target variable is **Exited** and there are remaining 13 variables.
- All the remaining 13 variables are not useful for prediction. We have to select the relevant variables.
- So, first let's check the variables in the dataset.

```
df.columns
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
      'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
      'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

Not all columns affect the customer churn. Let's discuss each column one by one:

1. **RowNumber** — corresponds to the record (row) number and has no effect on the output. This column will be removed.
2. **CustomerId** — contains random values and has no effect on customer leaving the bank. This column will be removed.
3. **Surname** — the surname of a customer has no impact on their decision to leave the bank. This column will be removed.

4. **CreditScore** — can have an effect on customer churn, since a customer with a higher credit score is less likely to leave the bank.
5. **Geography** — a customer's location can affect their decision to leave the bank. We'll keep this column.
6. **Gender** — it's interesting to explore whether gender plays a role in a customer leaving the bank. We'll include this column, too.
7. **Age** —this is certainly relevant, since older customers are less likely to leave their bank than younger ones.
8. **Tenure** — refers to the number of years that the customer has been a client of the bank. Normally, older clients are more loyal and less likely to leave a bank.
9. **Balance** — also a very good indicator of customer churn, as people with a higher balance in their accounts are less likely to leave the bank compared to those with lower balances.
10. **NumOfProducts** — refers to the number of products that a customer has purchased through the bank.
11. **HasCrCard** — denotes whether or not a customer has a credit card. This column is also relevant, since people with a credit card are less likely to leave the bank.
12. **IsActiveMember** — active customers are less likely to leave the bank, so we'll keep this.
13. **EstimatedSalary** — as with balance, people with lower salaries are more likely to leave the bank compared to those with higher salaries.
14. **Exited** — whether or not the customer left the bank. This is what we have to predict.

- So, we'll remove the **RowNumber**, **CustomerId** and **Surname** columns from our feature set. All the remaining columns do contribute to the customer churn in one way or another.
- We can drop these columns as follows:

```
df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1, inplace=True)
```

```
df.columns
```

```
Index(['CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance',
      'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary',
      'Exited'],
      dtype='object')
```

- We can see that the above three columns are removed from the dataset.

✓ 6. Convert categorical columns to numeric columns

- Now, in our dataset, we have two categorical columns: **Gender** and **Geography**. We should convert them into numerical format.

✓ 6.1 Explore Gender variable

```
df['Gender'].value_counts()
```

```

Gender
Male    5457
Female  4543

dtype: int64
```

- Here 1 stands for female and 0 stands for male.

✓ 6.2 Explore Geography variable

```
df['Geography'].value_counts()
```



	count
Geography	
France	5014
Germany	2509
Spain	2477

dtype: int64

df.head()



	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0



Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

Train-Test Split

7. Feature Scaling

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop(['Exited'], axis=1)
y = df['Exited']

# Identify categorical columns
categorical_features = ['Geography', 'Gender']
numerical_features = [col for col in X.columns if col not in categorical_features]

# Create a column transformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(drop='first'), categorical_features)])

# Split data into training and testing sets *before* applying the transformer
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

# Apply the column transformer to the training and testing data separately
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

# View the first few rows of the transformed training data (NumPy array)
print(X_train[:5])
```



[[0.35649971 -0.6557859 0.34567966 -1.21847056 0.80843615 0.64920267
0.97481699 1.36766974 0. 0. 1.]
[-0.20389777 0.29493847 -0.3483691 0.69683765 0.80843615 0.64920267
0.97481699 1.6612541 1. 0. 1.]
[-0.96147213 -1.41636539 -0.69539349 0.61862909 -0.91668767 0.64920267
-1.02583358 -0.25280688 0. 1. 1.]
[-0.94071667 -1.13114808 1.38675281 0.95321202 -0.91668767 0.64920267
-1.02583358 0.91539272 0. 0. 0.]
[-1.39733684 1.62595257 1.38675281 1.05744869 -0.91668767 -1.54035103
-1.02583358 -1.05960019 0. 0. 1.]]

8. Model Training

- We can see that our dataset is now ready to be fed into a ML model. We will proceed as follows:-

✓ 8.1 Predict accuracy with different algorithms

- Let's predict accuracy with different algorithms and evaluate their performance.

```
names = ["Logistic Regression", "Nearest Neighbors", "Naive Bayes", "Linear SVM", "RBF SVM",
         "Decision Tree", "Random Forest", "AdaBoost", "Gradient Boosting",
         "LDA", "QDA", "Neural Net", "XGBoost" ]
```

```
classifiers = [
    LogisticRegression(),
    KNeighborsClassifier(5),
    GaussianNB(),
    SVC(kernel="linear", C=0.025),
    SVC(kernel = "rbf", gamma=2, C=1),
    DecisionTreeClassifier(max_depth=5),
    RandomForestClassifier(max_depth=5, n_estimators=10, max_features=1),
    AdaBoostClassifier(),
    GradientBoostingClassifier(),
    LinearDiscriminantAnalysis(),
    QuadraticDiscriminantAnalysis(),
    MLPClassifier(alpha=1, max_iter=1000),
    xgb.XGBClassifier()
]
```

```
accuracy_scores = []
```

```
# iterate over classifiers and predict accuracy
for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)
    score = round(score, 4)
    accuracy_scores.append(score)
    print(name, ' : ', score)
```

```

Logistic Regression : 0.811
Nearest Neighbors : 0.845
Naive Bayes : 0.8335
Linear SVM : 0.8035
RBF SVM : 0.8115
Decision Tree : 0.858
Random Forest : 0.8195
AdaBoost : 0.8585
Gradient Boosting : 0.8675
LDA : 0.8085
QDA : 0.837
Neural Net : 0.864
XGBoost : 0.8695
```

```
classifiers_performance = pd.DataFrame({"Classifiers": names, "Accuracy Scores": accuracy_scores})
classifiers_performance
```



	Classifiers	Accuracy Scores	
0	Logistic Regression	0.8110	
1	Nearest Neighbors	0.8450	
2	Naive Bayes	0.8335	
3	Linear SVM	0.8035	
4	RBF SVM	0.8115	
5	Decision Tree	0.8580	
6	Random Forest	0.8195	
7	AdaBoost	0.8585	
8	Gradient Boosting	0.8675	
9	LDA	0.8085	
10	QDA	0.8370	
11	Neural Net	0.8640	
12	XGBoost	0.8695	

Next steps: [Generate code with classifiers_performance](#) [View recommended plots](#) [New interactive sheet](#)

- The accuracy score of top performing algorithms in descending order is given below -

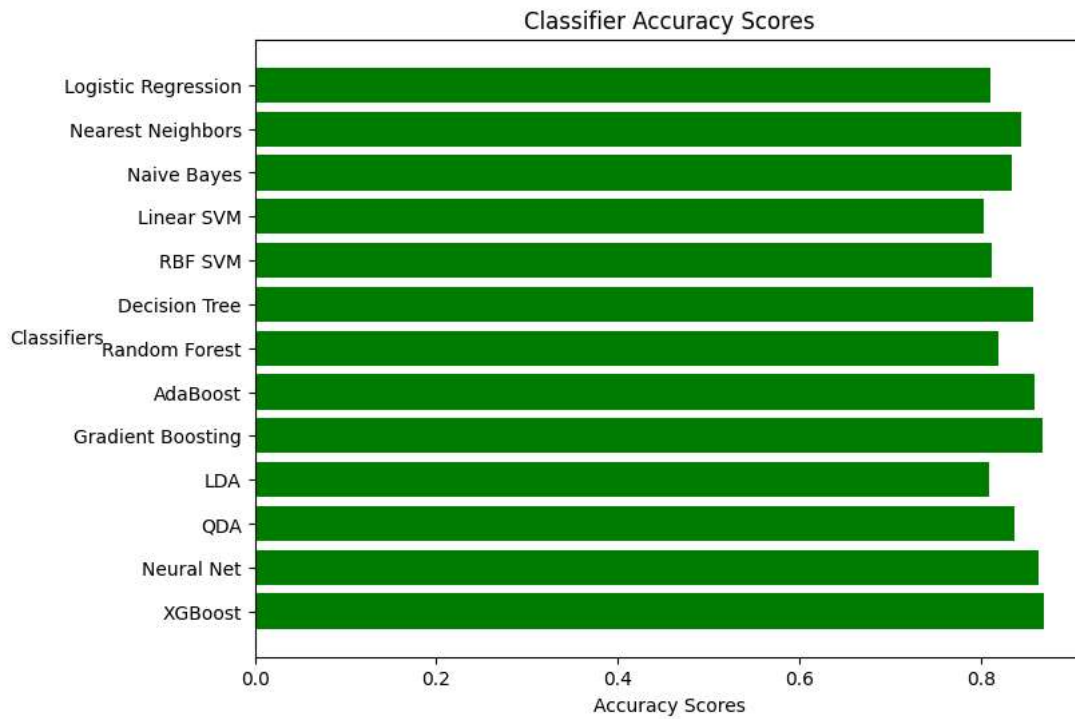
```
classifiers_performance.sort_values(by = 'Accuracy Scores' , ascending = False)[['Classifiers', 'Accuracy Scores']]
```



	Classifiers	Accuracy Scores	
12	XGBoost	0.8695	
8	Gradient Boosting	0.8675	
11	Neural Net	0.8640	
7	AdaBoost	0.8585	
5	Decision Tree	0.8580	
1	Nearest Neighbors	0.8450	
10	QDA	0.8370	
2	Naive Bayes	0.8335	
6	Random Forest	0.8195	
4	RBF SVM	0.8115	
0	Logistic Regression	0.8110	
9	LDA	0.8085	
3	Linear SVM	0.8035	

✓ 8.2 Plot the classifier accuracy scores

```
fig, ax = plt.subplots(figsize=(8,6))
x = classifiers_performance['Accuracy Scores']
y = classifiers_performance['Classifiers']
ax.barh(y, x, align='center', color='green')
ax.invert_yaxis() # labels read top-to-bottom
ax.set_xlabel('Accuracy Scores')
ax.set_ylabel('Classifiers', rotation=0)
ax.set_title('Classifier Accuracy Scores')
plt.show()
```



9. Feature Importance

- In this section, we will see how to improve model performance by feature selection.
- We will visualize feature importance with random forest classifier and drop the least important feature, rebuild the model and check effect on accuracy.
- For a comprehensive overview on feature selection techniques, please see the kernel -

[A Reference Guide to Feature Selection Methods](#)

9.1 Feature importance with Random Forest model

- Until now, I have used all the features given in the model. Now, I will select only the important features, build the model using these features and see its effect on accuracy.
- First, I will create the Random Forest model as follows:-

```
# instantiate the classifier with n_estimators = 100
clf = RandomForestClassifier(n_estimators=100, random_state=0)
```

```
# fit the classifier to the training set
clf.fit(X_train, y_train)
```



```
RandomForestClassifier
RandomForestClassifier(random_state=0)
```

- Now, I will use the feature importance variable to see feature importance scores.

```
# view the feature scores
feature_scores = pd.Series(clf.feature_importances_, index=preprocessor.get_feature_names_out()).sort_values(ascending=False)
feature_scores
```



```

0
num__Age      0.235508
num__EstimatedSalary  0.146218
num__CreditScore  0.142338
num__Balance     0.141445
num__NumOfProducts  0.134758
num__Tenure       0.081713
num__IsActiveMember  0.041586
cat__Geography_Germany  0.025667
num__HasCrCard     0.018422
cat__Gender_Male    0.018354
cat__Geography_Spain  0.013991

dtype: float64

from sklearn.model_selection import GridSearchCV
import xgboost as xgb

# Define the parameter grid to search
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Instantiate the XGBoost classifier
xgb_clf = xgb.XGBClassifier(random_state=42)

# Instantiate the GridSearchCV object
grid_search = GridSearchCV(estimator=xgb_clf, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)

# Fit the grid search to the training data
grid_search.fit(X_train, y_train)

# Print the best parameters and the best score
print("Best parameters found: ", grid_search.best_params_)
print("Best cross-validation accuracy score: {:.4f}".format(grid_search.best_score_))

# Evaluate the best model on the test data
best_xgb_model = grid_search.best_estimator_
test_accuracy = best_xgb_model.score(X_test, y_test)
print("Test accuracy with best model: {:.4f}".format(test_accuracy))

Best parameters found: {'colsample_bytree': 0.8, 'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200, 'subsample': 0.8}
Best cross-validation accuracy score: 0.8635
Test accuracy with best model: 0.8700

```

✓ Hyperparameter Tuning with GridSearchCV (XGBoost)

- We will use GridSearchCV to find the best hyperparameters for the XGBoost classifier to potentially improve its accuracy.
- We can see that the most important feature is **Age** and least important feature is **Gender**.

✓ 9.2 Drop least important feature

- Now, I will drop the least important feature **Gender** from the model, rebuild the model and check its effect on accuracy.

```

# Get the feature names after transformation
transformed_feature_names = preprocessor.get_feature_names_out()

# Find the index of the 'Gender_Male' column
try:

```

```

gender_male_index = list(transformed_feature_names).index('cat_Gender_Male')
except ValueError:
    print("'Gender_Male' column not found in transformed features.")
    gender_male_index = -1 # Set to -1 or handle as appropriate if the column is not found

# Drop the 'Gender_Male' column from X_train and X_test
if gender_male_index != -1:
    X1_train = np.delete(X_train, gender_male_index, axis=1)
    X1_test = np.delete(X_test, gender_male_index, axis=1)
    print("Shape of X1_train after dropping 'Gender_Male':", X1_train.shape)
    print("Shape of X1_test after dropping 'Gender_Male':", X1_test.shape)
else:
    X1_train = X_train
    X1_test = X_test
    print("'Gender_Male' column was not dropped as it was not found.")

↳ Shape of X1_train after dropping 'Gender_Male': (8000, 10)
   Shape of X1_test after dropping 'Gender_Male': (2000, 10)

```

```
accuracy_scores1 = []
```

```

# iterate over classifiers and predict accuracy
for name, clf in zip(names, classifiers):
    clf.fit(X1_train, y_train)
    score = clf.score(X1_test, y_test)
    score = round(score, 4)
    accuracy_scores1.append(score)
    print(name, ' : ', score)

```

```

↳ Logistic Regression : 0.8185
   Nearest Neighbors : 0.85
   Naive Bayes : 0.8335
   Linear SVM : 0.8035
   RBF SVM : 0.8205
   Decision Tree : 0.858
   Random Forest : 0.8345
   AdaBoost : 0.854
   Gradient Boosting : 0.867
   LDA : 0.8185
   QDA : 0.836
   Neural Net : 0.8625
   XGBoost : 0.86

```

- Now, we will compare our original accuracy and latest accuracy from the model.

```

classifiers_performance1 = pd.DataFrame({"Classifiers": names, "Accuracy Scores": accuracy_scores,
                                         "Accuracy Scores1": accuracy_scores1})
classifiers_performance1=classifiers_performance1.sort_values(by = 'Accuracy Scores' , ascending = False)
classifiers_performance1

```

	Classifiers	Accuracy Scores	Accuracy Scores1	
12	XGBoost	0.8695	0.8600	
8	Gradient Boosting	0.8675	0.8670	
11	Neural Net	0.8640	0.8625	
7	AdaBoost	0.8585	0.8540	
5	Decision Tree	0.8580	0.8580	
1	Nearest Neighbors	0.8450	0.8500	
10	QDA	0.8370	0.8360	
2	Naive Bayes	0.8335	0.8335	
6	Random Forest	0.8195	0.8345	
4	RBF SVM	0.8115	0.8205	
0	Logistic Regression	0.8110	0.8185	
9	LDA	0.8085	0.8185	
3	Linear SVM	0.8035	0.8035	

Next steps:

[Generate code with classifiers_performance1](#)[View recommended plots](#)[New interactive sheet](#)

- We can see that dropping the least important feature **Gender** from the model does not result in performance improvement.
- We can see that XGBoost has maximum accuracy of 0.866.
- So, we will use the XGBoost Classifier to plot the confusion-matrix.

✓ 10. Confusion Matrix

- A **confusion matrix** is a tool for summarizing the performance of a classification algorithm. A confusion matrix will give us a clear picture of classification model performance and the types of errors produced by the model. It gives us a summary of correct and incorrect predictions broken down by each category. The summary is represented in a tabular form.
- Four types of outcomes are possible while evaluating a classification model performance. These four outcomes are described below:-
- **True Positives (TP)** – True Positives occur when we predict an observation belongs to a certain class and the observation actually belongs to that class.
- **True Negatives (TN)** – True Negatives occur when we predict an observation does not belong to a certain class and the observation actually does not belong to that class.
- **False Positives (FP)** – False Positives occur when we predict an observation belongs to a certain class but the observation actually does not belong to that class. This type of error is called Type I error.
- **False Negatives (FN)** – False Negatives occur when we predict an observation does not belong to a certain class but the observation actually belongs to that class. This is a very serious error and it is called Type II error.
- These four outcomes are summarized in a confusion matrix.
- We will use the XGBoost Classifier to plot the confusion-matrix.

```
# instantiate the XGBoost classifier
xgb_clf = xgb.XGBClassifier()
```

```
# fit the classifier to the modified training set
xgb_clf.fit(X_train, y_train)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=None,
               n_jobs=None, num_parallel_tree=None, ...)
```

```
# predict on the test set
y_pred = xgb_clf.predict(X_test)
```

```
# print the accuracy
print('XGBoost Classifier model accuracy score: {0:0.4f}'.format(accuracy_score(y_test, y_pred)))
```

```
XGBoost Classifier model accuracy score: 0.8695
```

```
# print confusion-matrix
```

```
cm = confusion_matrix(y_test, y_pred)

print('Confusion matrix\n\n', cm)

print('\nTrue Positives(TP) = ', cm[0,0])

print('\nTrue Negatives(TN) = ', cm[1,1])

print('\nFalse Positives(FP) = ', cm[0,1])

print('\nFalse Negatives(FN) = ', cm[1,0])
```

↗ Confusion matrix

```
[[1521  86]
 [ 175 218]]
```

True Positives(TP) = 1521

True Negatives(TN) = 218

False Positives(FP) = 86

False Negatives(FN) = 175

The confusion matrix shows $1521 + 175 = 1732$ correct predictions and $86 + 218 = 304$ incorrect predictions.

In this case, we have

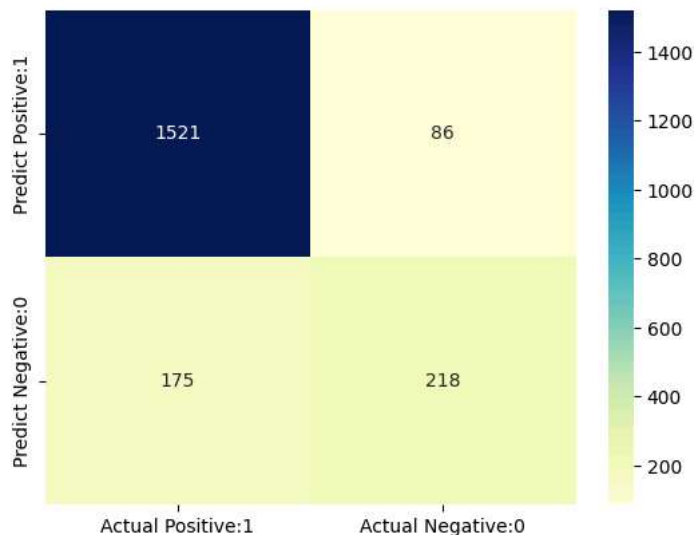
- True Positives (Actual Positive:1 and Predict Positive:1) - 1521
- True Negatives (Actual Negative:0 and Predict Negative:0) - 218
- False Positives (Actual Negative:0 but Predict Positive:1) - 86 (Type I error)
- False Negatives (Actual Positive:1 but Predict Negative:0) - 175 (Type II error)

visualize confusion matrix with seaborn heatmap

```
cm_matrix = pd.DataFrame(data=cm, columns=['Actual Positive:1', 'Actual Negative:0'],
                        index=['Predict Positive:1', 'Predict Negative:0'])
```

```
sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')
```

↗ <Axes: >



11. Classification Metrics

11.1 Classification Report

- **Classification Report** is another way to evaluate the classification model performance.
- It displays the **precision**, **recall**, **f1** and **support** scores for the model.
- We can print a classification report as follows:-

```
print(classification_report(y_test, y_pred))
```

↗

	precision	recall	f1-score	support
0	0.90	0.95	0.92	1607
1	0.72	0.55	0.63	393
accuracy			0.87	2000
macro avg	0.81	0.75	0.77	2000
weighted avg	0.86	0.87	0.86	2000

11.2 Classification Accuracy

```
TP = cm[0,0]
TN = cm[1,1]
FP = cm[0,1]
FN = cm[1,0]

# print classification accuracy

classification_accuracy = (TP + TN) / float(TP + TN + FP + FN)

print('Classification accuracy : {0:0.4f}'.format(classification_accuracy))
```

↗ Classification accuracy : 0.8695

11.3 Classification Error

```
# print classification error

classification_error = (FP + FN) / float(TP + TN + FP + FN)

print('Classification error : {0:0.4f}'.format(classification_error))
```

↗ Classification error : 0.1305

11.4 Precision

- **Precision** can be defined as the percentage of correctly predicted positive outcomes out of all the predicted positive outcomes. It can be given as the ratio of true positives (TP) to the sum of true and false positives (TP + FP).
- So, Precision identifies the proportion of correctly predicted positive outcome. It is more concerned with the positive class than the negative class.
- Mathematically, precision can be defined as the ratio of TP to (TP + FP).

```
# print precision score

precision = TP / float(TP + FP)

print('Precision : {0:0.4f}'.format(precision))
```

↗ Precision : 0.9465

11.5 Recall

- **Recall** can be defined as the percentage of correctly predicted positive outcomes out of all the actual positive outcomes. It can be given as the ratio of true positives (TP) to the sum of true positives and false negatives (TP + FN).
- **Recall** is also called **Sensitivity**.
- Recall identifies the proportion of correctly predicted actual positives.
- Mathematically, Recall can be given as the ratio of TP to (TP + FN).

```
recall = TP / float(TP + FN)

print('Recall or Sensitivity : {0:0.4f}'.format(recall))
```

↗ Recall or Sensitivity : 0.8968

11.6 True Positive Rate

- **True Positive Rate** is synonymous with **Recall**.

```
true_positive_rate = TP / float(TP + FN)

print('True Positive Rate : {0:0.4f}'.format(true_positive_rate))
```

True Positive Rate : 0.8968

11.7 False Positive Rate

```
false_positive_rate = FP / float(FP + TN)

print('False Positive Rate : {0:0.4f}'.format(false_positive_rate))
```

False Positive Rate : 0.2829

11.8 Specificity (True Negative Rate)

- **Specificity** is also called **True Negative Rate**.

```
specificity = TN / (TN + FP)

print('Specificity : {0:0.4f}'.format(specificity))
```

Specificity : 0.7171

11.9 f1-score

- **f1-score** is the weighted harmonic mean of precision and recall.
- The best possible f1-score would be 1.0 and the worst would be 0.0.
- f1-score is the harmonic mean of precision and recall.
- So, f1-score is always lower than accuracy measures as they embed precision and recall into their computation.
- The weighted average of f1-score should be used to compare classifier models, not global accuracy.

11.10 Support

- **Support** is the actual number of occurrences of the class in our dataset.

12. Cross Validation

- We will check whether **Cross Validation** results in performance improvement.

```
# iterate over classifiers and calculate cross-validation score
for name, clf in zip(names, classifiers):
    scores = cross_val_score(clf, X_train, y_train, cv = 10, scoring='accuracy')
    print(name , '{:0.4f}'.format(scores.mean()))
```

Logistic Regression :0.8089
 Nearest Neighbors :0.8351
 Naive Bayes :0.8227
 Linear SVM :0.7945
 RBF SVM :0.8010
 Decision Tree :0.8541
 Random Forest :0.8093
 AdaBoost :0.8521
 Gradient Boosting :0.8628
 LDA :0.8076
 QDA :0.8316
 Neural Net :0.8596
 XGBoost :0.8516

- It can be seen that cross-validation does not result in performance improvement.

13. Results and Conclusion

- In this project, we build an ML model to predict the churn-rate of the customer.

